IIT (ISM) Dhanbad

Department of Computer Science and Engineering

Project 2.3

Multithreaded File Management System on xv6

Using the Producer-Consumer Model

Course: Operating Systems Lab (NSCS210)

Instructor: Dr. Jaishree Mayank

Team Members

Prathamesh Vispute(24JE0674)
Pranjul(24JE0673)
Prathyush P(24JE0675)
Punit(24JE0676)
Poka Sireesha(24JE0671)
Prabir Soren(24JE0672)

1. Introduction

This report describes a concurrent file management system implemented on the xv6 operating system using the producer-consumer model. The goal of the project is to allow multiple file operations to execute safely and concurrently while preventing data corruption through proper synchronization.

The project requirements include support for concurrent reading, exclusive writing, file deletion, renaming, copying, metadata display, error handling, operation logging, and compression/decompression of files.

xv6 is a minimal teaching operating system and does not provide a full modern multithreading library like pthreads in user space. To work within this constraint and keep the implementation stable, multiple worker processes created using fork() are used to simulate concurrent workers. These workers consume file-operation tasks produced by a main producer process. Communication between the producer and consumers is handled through pipes, and synchronization is enforced using custom kernel-level file locks added as new system calls.

2. GithHub Repo Link:

https://github.com/PrathamandX/G15_Project2_3

2. Objectives

The main objectives of this project are:

- Implement a file management system on xv6.
- Support concurrent file operations.
- Demonstrate the producer-consumer model for task distribution.
- Enforce safe synchronization for shared file access.
- Provide proper logging and error handling.
- Demonstrate file compression and decompression.
- Gain practical understanding of how operating systems handle concurrency, file management, and synchronization.

3. Problem Statement

The project requires implementation of a concurrent file management system with the following nine key features:

1. Concurrent file reading
2. Exclusive file writing
3. File deletion
4. File renaming
5. File copying
6. File metadata display
7. Robust error handling
8. Logging of operations
9. Compression and decompression of files

All nine features are implemented and tested in this project as described in the sections below.

4. Design Approach

4.1 Why xv6

xv6 is a simplified UNIX-like teaching operating system. It is small enough to understand and modify, making it well-suited for OS lab projects that involve kernel changes, system calls, process management, and synchronization.

4.2 Producer-Consumer Model

The producer-consumer model is a natural fit for managing file operation tasks:

- The producer accepts a file operation request and places it into a task queue.
- Consumer workers pick up tasks from the queue and execute them.
- This allows multiple file tasks to be handled concurrently.

In this implementation:

- The producer is the main fmops process that parses the command-line request.
- The consumers are worker processes created by fork().
- The queue is implemented using pipes.
- Synchronization is provided by custom file-locking system calls.

4.3 Processes Instead of Threads

The project specification mentions threads and synchronization. However, xv6 does not provide a complete pthread-style thread library in user space by default. To keep the implementation minimal and stable on xv6, multiple concurrent worker processes created via fork() are used instead of user-level threads.

This approach still demonstrates all the required concepts: concurrent execution, producer-consumer behavior, synchronization, and safe shared-resource access.

5. System Architecture

The system is divided into two layers: a kernel layer and a user layer.

5.1 Kernel Layer

The xv6 kernel was extended with a small lock manager that supports per-file locking. Two custom system calls were added:

- fmlock(path, mode) — acquires a lock on a specified file.
- fmunlock(path, mode) — releases a lock on a specified file.

These syscalls enforce the following access rules:

- Multiple readers can hold a shared lock on the same file simultaneously.
- Only one writer can modify a file at a time.
- A writer cannot acquire the lock if any readers are active.
- No other process can access the file while a writer holds the lock.

5.2 User Layer

The user program fmops performs the actual file operations. When invoked from the xv6 shell, it:

1. Parses the command-line arguments to determine the requested operation.
2. Creates worker processes.
3. Sends the task to a consumer worker through a pipe.
4. The consumer executes the operation with appropriate locking.
5. Results are logged to fmops.log.

6. Features Implemented

6.1 Concurrent File Reading

Multiple worker processes can read the same file concurrently. Each read operation acquires a shared read lock before accessing the file, ensuring safe concurrent access.

6.2 Exclusive File Writing

Write operations acquire an exclusive write lock, ensuring that only one process can modify a file at a time. Other operations on the same file are blocked until the write completes.

6.3 File Deletion

A file is deleted safely only when no conflicting lock is active on it. The delete operation acquires an exclusive write lock before unlinking the file.

6.4 File Renaming

Rename is implemented safely using file locking. Since the base version of xv6 does not provide a native rename system call, the operation is performed using link() followed by unlink().

6.5 File Copying

Copy reads the source file and writes to the destination file using coordinated locking. A shared read lock is held on the source and an exclusive write lock on the destination.

6.6 File Metadata Display

Metadata displayed for a file includes:

- File type
- Inode number
- Link count
- File size

6.7 Error Handling

Each operation checks for common failure conditions such as:

- File does not exist.
- Invalid source or destination path.
- Delete, rename, or copy failure.
- Compression or decompression failure.

Errors are reported to standard output and are also written to the log file.

6.8 Logging

All operations are appended to a log file fmops.log. Each log entry records the following fields:

- tick — current uptime in xv6 ticks.
- pid — process ID of the worker.
- op — operation name.
- a — primary file.
- b — secondary file (if applicable).
- rc — return code, 0 for success and -1 for failure.

A sample log entry looks like:

```
tick=123 pid=5 op=copy a=src1 b=dst1 rc=0
```

Logging is also protected with a write lock so concurrent log writes do not corrupt the log file.

6.9 Compression and Decompression

A simple run-length encoding (RLE) method is used for compression. Input data is encoded as pairs of (count, character). Decompression reverses the process to restore the original content.

Example: The input string aaaaabbbbcccccc is encoded as 5a4b6c.

RLE was chosen because it is simple to implement on xv6, sufficient for demonstrating concurrent file transformation, and requires no external libraries.

7. File Structure and Modifications

7.1 New Files Added

- kernel/fmlock.c — kernel-side lock manager implementation.
- user/fm.c — user-space file management program.

7.2 Existing Files Modified

- kernel/defs.h
- kernel/main.c
- kernel/syscall.h
- kernel/syscall.c
- kernel/sysfile.c
- user/user.h
- user/usys.pl
- Makefile

8. Kernel Changes

8.1 File Lock Table

A kernel-level lock table stores file lock information. Each entry contains the file name, a reader count, a writer flag, and a usage status bit.

8.2 Locking Strategy

Two lock modes are supported: FM_READ and FM_WRITE.

The locking rules are:

- Multiple readers are allowed simultaneously.
- Only one writer is allowed at a time.
- A writer cannot proceed if any readers are active.
- No other access is allowed while a writer holds the lock.

8.3 System Calls Added

fmlock(char *name, int mode)

Acquires a lock on the specified file in either FM_READ or FM_WRITE mode.

fmunlock(char *name, int mode)

Releases the lock on the specified file. These calls allow user-level file operations to coordinate safely through the kernel without race conditions.

9. User Program Design

The program fmops is the user-space file manager. It is invoked from the xv6 shell with the following syntax:

```
fmops read <file>
fmops write <file> <text>
fmops copy <src> <dst>
fmops rename <old> <new>
fmops delete <file>
fmops meta <file>
fmops compress <file>
fmops decompress <file.rle>
```

The internal execution flow is:

6. The xv6 shell starts fmops and passes command-line arguments.
7. fmops parses the arguments to identify the operation.
8. fmops creates worker processes via fork().
9. The producer writes the task into a pipe.
10. One worker reads from the pipe and executes the operation.
11. The operation acquires the appropriate lock, executes, and releases the lock.
12. The result is written to fmops.log.
13. Worker processes exit and the parent process waits for them to finish.

10. Synchronization Strategy

Synchronization is the core requirement of this project. The correct use of read and write locks ensures that concurrent file operations do not corrupt shared data.

10.1 Operations Using Read Lock

- Read

- Metadata display
- Source side of copy
- Source side of compression

10.2 Operations Using Write Lock

- Write
- Delete
- Rename
- Destination side of copy
- Destination side of decompression
- Log file updates

10.3 Benefits

This strategy prevents:

- Simultaneous writes corrupting a file.
- Deletion while a file is being read or written.
- Rename operations conflicting with active reads or writes.
- Partial or mixed content from concurrent writes.

11. Build and Execution

11.1 Build

From the xv6 root directory:

```
make clean
make qemu
```

11.2 Create Sample Files

Inside the xv6 shell:

```
echo hello_world > a
echo operating_systems_lab > b
echo aaaaabbbbcccccc > comp
```

11.3 Run Commands

```
fmops read a
fmops write a new_content
fmops copy a c
fmops rename c d
fmops meta d
fmops compress comp
fmops decompress comp.rle
fmops delete d
cat fmops.log
```

12. Testing

This section presents test cases for all nine required features. Each test specifies the purpose, commands, expected output, and pass/fail criteria.

Test 1: Concurrent File Reading

Purpose: Verify that multiple workers can read the same file safely at the same time.

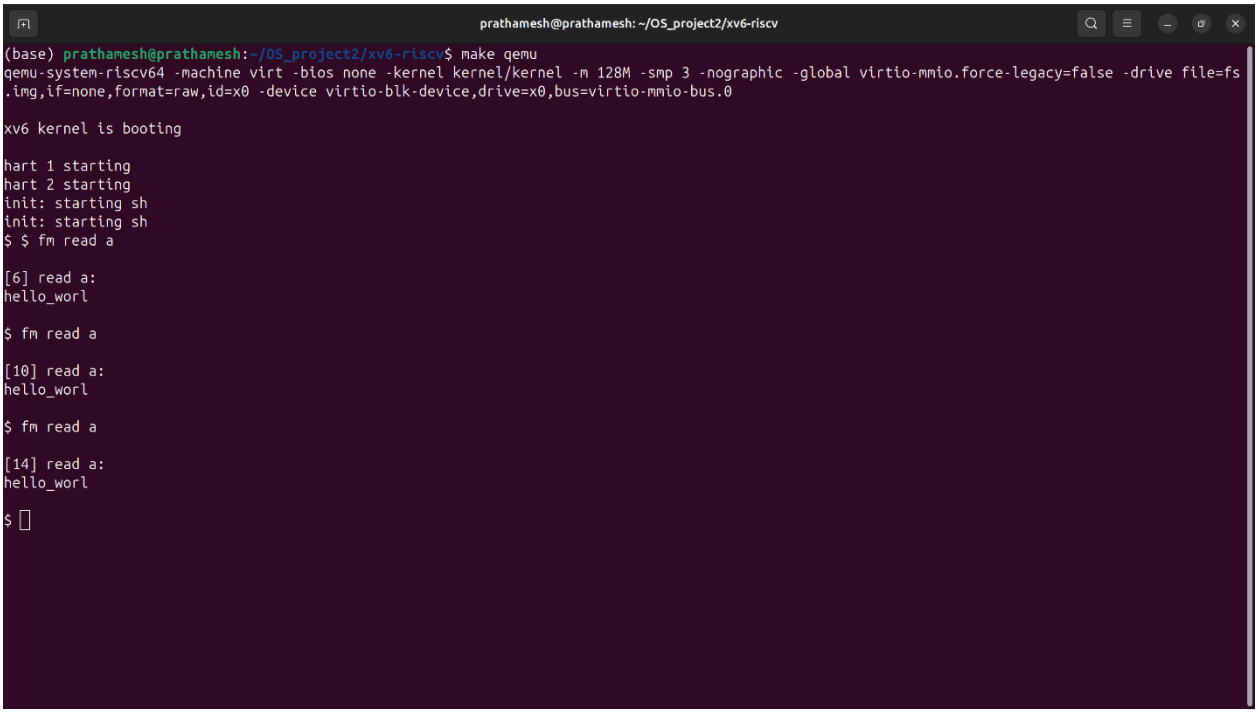
Commands

```
fmops read a
fmops read a
fmops read a
```

Expected Output

```
hello_world
```

Pass if: All reads succeed, file content remains unchanged, and no corruption occurs.



Test 2: Exclusive File Writing

Purpose: Verify that write operations are serialized and the file ends up with a clean final value.

Commands

```
fmops write a first_write
fmops write a second_write
fmops read a
```

Expected Output

```
second_write
```

Pass if: The file contains exactly one clean value with no mixed or corrupted text.

```
[14] read a:
hello_worl

$ fm write a first_write
$ fm write a second_write
$ fm read a

[26] read a:
second_write

$
```

Test 3: File Deletion

Purpose: Verify that a file is deleted safely and becomes inaccessible afterward.

Commands

```
echo delete_me > del1
fmops read del1
fmops delete del1
fmops read del1
```

Expected Output

The first read succeeds. The delete succeeds. The second read reports an error.

Pass if: The deleted file cannot be read after deletion.

```
$ echo delete_me >d1
$ fm read d1

[31] read d1:
delete_me

$ fm delete d1
$ fm read d1
usage:
  fm read <file>
  fm write <file> <text>
  fm copy <src> <dst>
  fm rename <old> <new>
  fm delete <file>
  fm meta <file>
  fm compress <file>
  fm decompress <file.rle>
$
```

Test 4: File Renaming

Purpose: Verify that a file is correctly renamed and accessible under the new name only.

Commands

```
echo rename_test > old1
fmops rename old1 new1
fmops read old1
fmops read new1
```

Expected Output

Reading old1 reports an error. Reading new1 produces:

```
rename_test
```

Pass if: New name works and the old name is no longer accessible.

Test 5: File Copying

Purpose: Verify that copying produces an independent duplicate of the source file.

Commands

```
echo copy_content > src1
fmops copy src1 dst1
fmops read src1
fmops read dst1
```

Expected Output

Both files contain:

```
copy_content
```

Additional Validation

```
fmops write dst1 changed_copy
fmops read src1
fmops read dst1
```

Expected: src1 remains unchanged; dst1 shows the new content.

Pass if: The destination receives the correct copied content and modifications to the copy do not affect the original.

Test 6: Metadata Display

Purpose: Verify that file metadata is retrieved and displayed correctly.

Commands

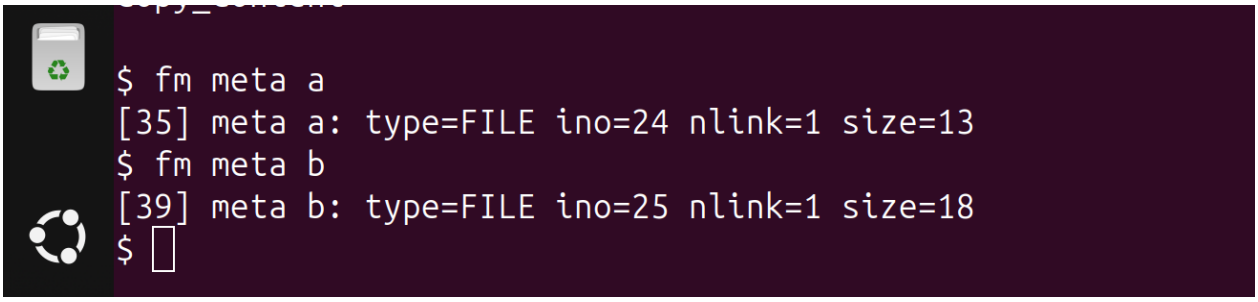
```
fmops meta a
fmops meta b
```

Expected Output

```
meta a: type=FILE ino=18 nlink=1 size=12
```

Note: Metadata available in xv6 is limited to the fields provided by the stat syscall. Creation timestamps and Unix-style permissions are not available in the base xv6 version.

Pass if: Type, inode number, link count, and file size are displayed correctly.



Test 7: Error Handling

Purpose: Verify that invalid operations fail gracefully without crashing the system.

Commands

```
fmops read nofile
fmops delete nofile
fmops rename missing target
fmops copy missing x
fmops decompress nothing.rle
```

Expected Output

Each command reports an appropriate error message.

Pass if: No system crash occurs, each operation fails cleanly, and the failure is recorded in fmops.log.

```
prathamesh@prathamesh: ~/OS_project2/xv6-riscv
(base) prathamesh@prathamesh:~/OS_project2/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs
.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
init: starting sh
$ $ echo hello_worl > a
$ echo Operating_systems > b
$ echo abbdkddskdd; > comp
abbdkddskdd
$ fm read a

[10] read a:
hello_worl

$ fm read b

[14] read b:
Operating_systems

$ fm read comp
usage:
  fm read <file>
  fm write <file> <text>
  fm copy <src> <dst>
  fm rename <old> <new>
  fm delete <file>
  fm meta <file>
  fm compress <file>
  fm decompress <file.rle>
$
```

Test 8: Logging

Purpose: Verify that all operations are recorded in the log file.

Commands

```
fmops read a
fmops write b new_data_for_b
fmops copy a c
fmops meta c
cat fmops.log
```

Expected Output

The log file should contain entries similar to:

```
tick=... pid=... op=read  a=a b=- rc=0
tick=... pid=... op=write a=b b=- rc=0
tick=... pid=... op=copy  a=a b=c rc=0
tick=... pid=... op=meta  a=c b=- rc=0
```

Pass if: Log entries exist for all operations, including both successful and failed ones.

```
prathamesh@prathamesh: ~/OS_project2/xv6-riscv
$ cat fm.log
tick=2424 pid=35 op=read a=a b= rc=0
tick=2578 pid=39 op=read a=a b= rc=0
tick=2843 pid=43 op=write a=a b= rc=0
tick=3156 pid=49 op=read a=a b= rc=0
tick=3879 pid=54 op=read a=aldj b= rc=-1
tick=91 pid=6 op=read a=a b= rc=0
tick=561 pid=10 op=read a=a b= rc=0
tick=606 pid=14 op=read a=b b= rc=0
tick=54 pid=6 op=read a=a b= rc=0
tick=91 pid=10 op=read a=a b= rc=0
tick=127 pid=14 op=read a=a b= rc=0
tick=723 pid=18 op=write a=a b= rc=0
tick=796 pid=22 op=write a=a b= rc=0
tick=827 pid=26 op=read a=a b= rc=0
tick=1514 pid=31 op=read a=d1 b= rc=0
tick=1569 pid=35 op=delete a=d1 b= rc=0
tick=2236 pid=41 op=rename a=old1 b=new1 rc=0
tick=2291 pid=45 op=rename a=old1 b=new1 rc=-1
tick=2344 pid=49 op=rename a=old1 b=new1 rc=-1
tick=259 pid=8 op=read a=new1 b= rc=0
tick=304 pid=12 op=read a=old1 b= rc=0
tick=611 pid=16 op=read a=old1 b= rc=0
tick=1293 pid=21 op=copy a=src1 b=dst1 rc=0
tick=1645 pid=27 op=read a=src1 b= rc=0
tick=1712 pid=31 op=read a=dst1 b= rc=0
tick=2142 pid=35 op=meta a=a b= rc=0
tick=2170 pid=39 op=meta a=b b= rc=0
tick=48 pid=6 op=read a=nofile b= rc=-1
tick=136 pid=10 op=delete a=nofile b= rc=-1
tick=395 pid=15 op=read a=a b= rc=0
tick=584 pid=20 op=write a=wsafm b= rc=0
tick=644 pid=24 op=write a=b b= rc=0
tick=710 pid=28 op=meta a=b b= rc=0
$
```

Test 9: Compression and Decompression

Purpose: Verify that a file can be compressed and fully restored to its original content.

Commands

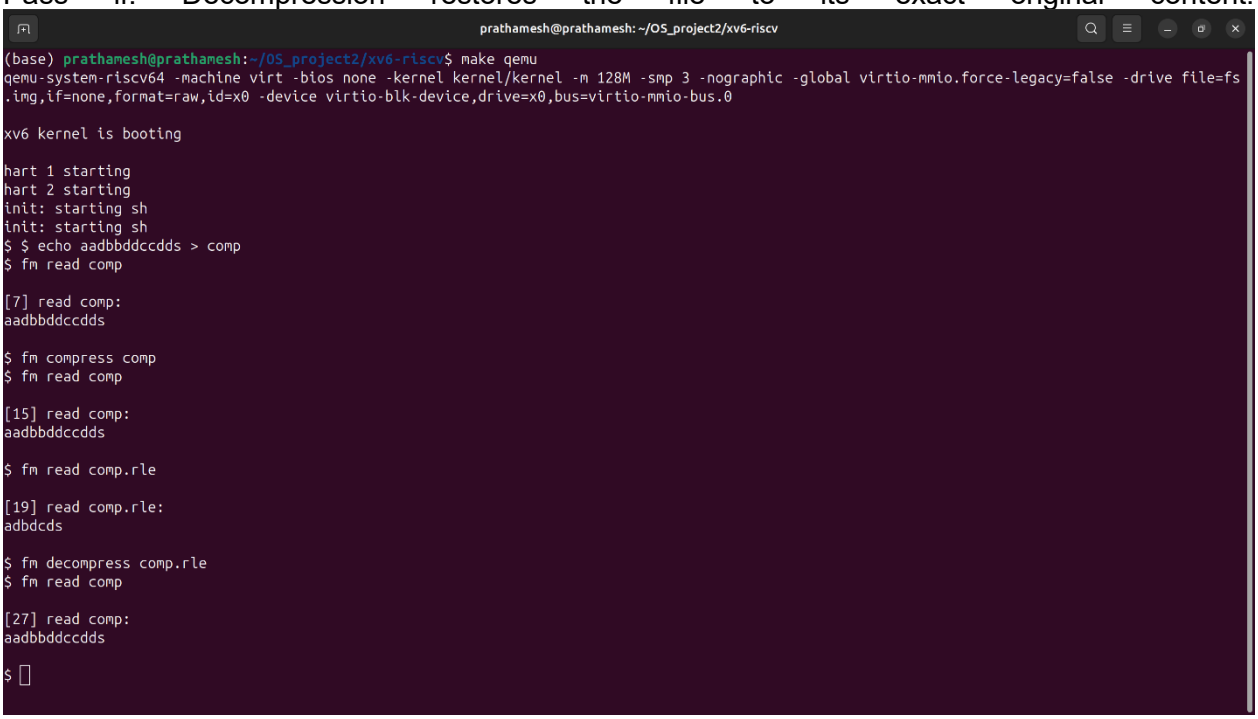
```
echo aaaaabbbbcccccc > comp
fmops read comp
fmops compress comp
fmops decompress comp.rle
fmops read comp
```

Expected Output

The final read of comp should match the original:

```
aaaaabbbbcccccc
```

Pass if: Decompression restores the file to its exact original content.



13. Combined Demonstration Sequence

The following command sequence can be used for demonstration or viva to cover all nine features in a single run:

```
echo hello_world > a
echo operating_systems_lab > b
echo aaaaabbbbcccccc > comp

fmops read a
fmops write a rewritten_data
fmops read a
fmops copy a c
fmops read c
fmops rename c d
fmops read d
fmops meta d
fmops compress comp
fmops decompress comp.rle
fmops read comp
fmops delete d
fmops read d
fmops read nofile
cat fmops.log
```

This sequence demonstrates reading, writing, copying, renaming, metadata display, compression, decompression, deletion, error handling, and logging in one continuous session.

14. Observations

During testing, the following behavior was observed:

- Multiple read operations completed safely without any file corruption.
- Write operations updated file contents correctly and without interference between concurrent requests.
- File deletion removed the target file and subsequent reads failed as expected.
- Rename worked correctly using the link + unlink approach.
- Copy preserved the original file and produced an accurate duplicate.
- Metadata display showed the correct inode, type, link count, and size fields.
- Invalid operations (missing files, bad paths) failed without crashing xv6.
- The log file recorded both successful and failed operations with correct details.
- Compression and decompression restored file data exactly.

15. Limitations

This implementation is intentionally minimal and has the following known limitations:

- Worker concurrency is implemented using processes (fork), not true pthread-style threads. This is a consequence of xv6 not supporting a full user-level thread library.
- Rename is implemented using link + unlink since xv6 does not have a native rename syscall.
- File metadata in xv6 is limited to fields available from the stat syscall; creation timestamps and permissions are not fully accessible.
- Compression uses simple run-length encoding rather than a general-purpose algorithm like gzip.
- Lock fairness is minimal; heavy sustained read traffic may delay writer processes.

Despite these limitations, the system correctly demonstrates all required features: concurrency, synchronization, file management, producer-consumer design, logging, error handling, and compression.

16. Learning Outcomes

Through this project, the team gained practical understanding of:

- How to add custom system calls to the xv6 kernel.
- How user space and kernel space interact in a UNIX-like operating system.
- How the producer-consumer model can be applied to file task management.
- How concurrent file access must be synchronized using reader-writer style locking.
- How logging and error handling improve the reliability and debuggability of concurrent systems.
- How kernel modifications can directly support user-level concurrency requirements.

17. Conclusion

This project implements a minimal but functional file management system on the xv6 operating system using a producer-consumer model. The system supports all nine required file operations and demonstrates safe concurrent access through kernel-level synchronization.

The custom fmlock and fmunlock system calls provide a readers-writer lock mechanism at the kernel level, which correctly coordinates access across concurrent worker processes. All operations are logged, errors are handled gracefully, and compression/decompression has been verified to restore file content accurately.

The project satisfies the requirements of Project 2.3 and serves as a practical demonstration of operating system concepts including concurrency, synchronization, file systems, and system call design on a real kernel codebase.

18. Contribution

Prathamesh : Implemented kernel-level file lock manager and syscall integration, testing and screenshots.
Pratyush: Implemented command parsing, producer-consumer flow, worker creation, and pipe-based task dispatch, and documentation.
Pranjul: Implemented concurrent read and exclusive write operations with synchronization.
Punit: Implemented file copy, rename, and delete operations.
Sireesha: Implemented metadata display, and error handling.
Prabir: Implemented compression/decompression and logging system.